

# Research Assistant Agent

Fetch papers, store embeddings, retrieve the right chunks, and summarize with RAGAS evaluation.

Stack: LangChain + ChromaDB + arXiv API + RAGAS

```
!pip install -qU arxiv pypdf langchain langchain-community langchain-chroma langchain-huggingface sentence-transformers langchain-groq datasets ragas litellm openai
```

```
!pip install -qU langchain-openai
```

Starting by importing `arxiv`

```
import arxiv

client = arxiv.Client()

search = arxiv.Search(
    query = "Agentic AI",
    max_results = 10,
    sort_by = arxiv.SortCriterion.SubmittedDate
)

results = client.results(search)

for r in client.results(search):
    print(r.title)
    print("\n")
    print(r.authors)
    print("\n")
    print(r.summary)
    print("\n\n")
```

Using `PyPDFLoader` to load the received research paper in the `docs` list

```
from langchain_community.document_loaders import PyPDFLoader
import arxiv

client = arxiv.Client()
search = arxiv.Search(
    query = "Agentic AI",
    max_results = 10,
    sort_by = arxiv.SortCriterion.SubmittedDate
)

docs = []
for result in client.results(search):
```



```

n", ".", "!", "?", ",", " ", ""]
)
parent_splitter = RecursiveCharacterTextSplitter(chunk_size=2000,
                                                  chunk_overlap=200,
                                                  separators=["\n\n",
"\n", ".", "!", "?", ",", " ", ""]
)

final_chunks = parent_splitter.split_documents(docs)
print(len(final_chunks))

770

```

## Using `HuggingFaceEmbeddings`

```

!pip install sentence-transformers

from langchain_chroma import Chroma
from langchain_huggingface import HuggingFaceEmbeddings

embeddings_model = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2",
    model_kwargs={'device': 'cpu'}
)

```

## Storing embeddings in `ChromaDB`

!Note:- Uncomment the below code only for approach 1

```

# vector_store = Chroma.from_documents(
#     documents = final_chunks,
#     embedding = embeddings_model,
#     persist_directory="./chroma_db"
# )

from langchain_core.stores import InMemoryStore
vectorstore = Chroma.from_documents(documents = final_chunks,
embedding=embeddings_model, persist_directory="./chroma_db")
store = InMemoryStore()

import os
from getpass import getpass
if "GROQ_API_KEY" not in os.environ:
    os.environ["GROQ_API_KEY"] = getpass("Enter your Groq API Key: ")

Enter your Groq API Key: .....

```

# For chunking strategy 1

```
retriever = vector_store.as_retriever(search_kwargs={"k": 3})
```

# For chunking strategy 2

## Custom Parent-Document Retrieval (Chunking Strategy 2)

This section implements a custom parent-document retrieval mechanism, replacing the direct use of LangChain's `MultiVectorRetriever` due to persistent import issues.

```
# Imports for custom parent-document retrieval (Chunking Strategy 2)
import uuid
from langchain_community.vectorstores import Chroma
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.stores import InMemoryStore
from langchain_core.runnables import RunnableLambda

# Define splitters for custom parent-document retrieval (Chunking Strategy 2)
parent_splitter_custom =
RecursiveCharacterTextSplitter(chunk_size=2000, chunk_overlap=200)
child_splitter_custom = RecursiveCharacterTextSplitter(chunk_size=400,
chunk_overlap=40)

# Setup a new Chroma Vector Store and an InMemoryStore for parent documents (Chunking Strategy 2)
# Note: This is separate from any previously defined 'vectorstore' using HuggingFaceEmbeddings.
vectorstore_custom = Chroma(collection_name="child_chunks_custom",
embedding_function=embeddings_model)
parent_document_store_custom = InMemoryStore()

/tmp/ipykernel_4158/3612182256.py:3: LangChainDeprecationWarning: The
class `Chroma` was deprecated in LangChain 0.2.9 and will be removed
in 1.0. An updated version of the class exists in the `langchain-
chroma` package and should be used instead. To use it run `pip install
-U `langchain-chroma` and import as `from `langchain_chroma import
Chroma`.
    vectorstore_custom = Chroma(collection_name="child_chunks_custom",
embedding_function=embeddings_model)

# Ingestion Loop for custom parent-document retrieval (Chunking Strategy 2)
# 'docs' is assumed to be loaded from previous cells (e.g., cell
`72qmDNmCMjZ-`)

print("Starting ingestion for custom parent-document retrieval...")
```

```

for doc in docs:
    # Break raw file into larger Parent Chunks
    parents = parent_splitter_custom.split_documents([doc])

    for parent_doc in parents:
        # Generate a completely unique ID for this specific parent
        chunk
        parent_id = str(uuid.uuid4())

        # Save the parent chunk to our dictionary store
        parent_document_store_custom.mset([(parent_id, parent_doc)])

        # Break that parent down into smaller Child Chunks
        children = child_splitter_custom.split_documents([parent_doc])

        # Tag every single child chunk metadata with its parent's ID
        for child_doc in children:
            child_doc.metadata["parent_id"] = parent_id

        # Add the child chunks to the vector database
        vectorstore_custom.add_documents(children)
print("Ingestion complete for custom parent-document retrieval.")

```

Starting ingestion for custom parent-document retrieval...

Ingestion complete for custom parent-document retrieval.

*# Custom retrieval function (Chunking Strategy 2)*

```

def custom_parent_retrieve(query, k=4):
    # Search the vector database for the closest tiny child chunks
    matching_children = vectorstore_custom.similarity_search(query,
k=k)
    print(f"Debug: Matching children found: {len(matching_children)}")

    retrieved_parents = []
    seen_parent_ids = set()

    for child in matching_children:
        # Extract the parent ID from the metadata
        p_id = child.metadata.get("parent_id")

        # If the parent ID exists and we haven't added it yet, fetch
        it
        if p_id and p_id not in seen_parent_ids:
            seen_parent_ids.add(p_id)
            # Use mget from InMemoryStore
            parent_doc_list =
parent_document_store_custom.mget([p_id])
            if parent_doc_list and parent_doc_list[0]: # Check if not
empty and doc exists
                retrieved_parents.append(parent_doc_list[0])

```

```

    print(f"Debug: Retrieved parents count: {len(retrieved_parents)}")
    return retrieved_parents

# Assign the custom retrieval logic to the 'retriever' variable for
# the RAG chain (Chunking Strategy 2)
# Using RunnableLambda to make the function compatible with
# LangChain's Runnable interface.
retriever = RunnableLambda(lambda query:
    custom_parent_retrieve(query))

!pip install langchain-experimental

# The following code was attempting to use MultiVectorRetriever,
# but has been replaced by the custom parent-document retrieval logic
# defined in cells related to 'Chunking Strategy 2 Alternative'.

# from langchain_experimental.retrievers.multi_vector import
# MultiVectorRetriever

# id_key = 'doc_id'
# retriever = MultiVectorRetriever(
#     vectorstore=vectorstore,
#     docstore=store,
# )

from langchain_groq import ChatGroq
llm = ChatGroq(model_name="llama3-70b-8192", temperature=0)

prompt_template = """You are an expert Research Assistant. Answer the
user's question using ONLY the provided research paper context.
If you do not know the answer based on the context, say that you don't
know.

Context:
{context}

Question: {question}

Answer: """

from langchain_core.prompts import ChatPromptTemplate
prompt = ChatPromptTemplate.from_template(prompt_template)

# helper function to format retrieved docs in a clean text block
def format_docs(docs):
    return "\n\n".join(doc.page_content for doc in docs)

from langchain_groq import ChatGroq
# llm = ChatGroq(model_name="mixtral-8x7b-32768", temperature=0)
llm = ChatGroq(model_name="llama-3.3-70b-versatile", temperature=0)

from langchain_core.output_parsers import StrOutputParser

```

```

from langchain_core.runnables import RunnablePassthrough

rag_chain = (
    {"context": retriever | format_docs, "question":
RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)

query = "What is RevengeBench? What is its purpose?"
response = rag_chain.invoke(query)

print(f"Query: {query}\n")
print(f"Answer:\n{response}")

Debug: Matching children found: 4
Debug: Retrieved parents count: 4
Query: What is RevengeBench? What is its purpose?

Answer:
REVENGEBENCH is a benchmark that measures whether a Large Language
Model (LLM) can infer the programmatic policy of another player from
behavioral evidence. Its purpose is to study the inverse direction of
policy generation, where instead of generating policies that play
well, it aims to recover a hidden policy's decision program from
behavioral traces and controlled interactions.

```

## RAGAs evaluation

### Bulding an evaluation dataset

```

import pandas as pd
from datasets import Dataset

# 1. Define your sample test questions
eval_questions = [
    "What are the core components of an Agentic AI architecture?",
    "What specific challenges or bottlenecks regarding Agentic AI are
discussed?",
    "How does memory play a role in agent frameworks according to the
papers?"
]

# 2. Loop through and capture matching metrics
compiled_data = []

for query in eval_questions:

```

```

print(f"Running RAG pipeline for query: '{query}'...")

# Fetch from your standard ChromaDB retriever
retrieved_chunks = retriever.invoke(query)
contexts_strings = [doc.page_content for doc in retrieved_chunks]

# Generate the live answer using your LangChain Groq pipeline
generated_answer = rag_chain.invoke(query)

# FIX: Update the dictionary keys to EXACTLY match RAGAS native
schemas
compiled_data.append({
    "user_input": query,
    "response": generated_answer,          # Changed 'answer' ->
'response'
    "retrieved_contexts": contexts_strings # Kept as
'retrieved_contexts'
})

# 3. Compile your target Hugging Face Dataset object
eval_dataset = Dataset.from_list(compiled_data)
print("\nEvaluation dataset rebuilt successfully with correct column
mapping keys!")

```

Running RAG pipeline for query: 'What are the core components of an Agentic AI architecture?'...

Debug: Matching children found: 4

Debug: Retrieved parents count: 4

Debug: Matching children found: 4

Debug: Retrieved parents count: 4

Running RAG pipeline for query: 'What specific challenges or bottlenecks regarding Agentic AI are discussed?'...

Debug: Matching children found: 4

Debug: Retrieved parents count: 4

Debug: Matching children found: 4

Debug: Retrieved parents count: 4

Running RAG pipeline for query: 'How does memory play a role in agent frameworks according to the papers?'...

Debug: Matching children found: 4

Debug: Retrieved parents count: 4

Debug: Matching children found: 4

Debug: Retrieved parents count: 4

Evaluation dataset rebuilt successfully with correct column mapping keys!



## We will run three primary metrics that do not require human ground-truth answers:

1. Faithfulness: Measures if the answer hallucinated or stayed entirely factual to the text chunks.
2. AnswerRelevance: Measures if the generated response actually answered the user's specific prompt.
3. ContextPrecision: Measures whether the retriever successfully put the most relevant chunks early in the ranking.

## Results from chunkning strategy 1

```
import sys
from types import ModuleType

# 1. Keep the VertexAI dummy module mock at the very top to bypass
that legacy bug
mock_vertex = ModuleType("langchain_community.chat_models.vertexai")
mock_vertex.ChatVertexAI = None
sys.modules["langchain_community.chat_models.vertexai"] = mock_vertex

mock_llm_vertex = ModuleType("langchain_community.llms.vertexai")
mock_llm_vertex.VertexAI = None
sys.modules["langchain_community.llms.vertexai"] = mock_llm_vertex

# =====
# RAGAS 0.4+ Modern Native Open-Source Infrastructure
# =====
from ragas import evaluate, EvaluationDataset
from ragas.llms import llm_factory
from ragas.embeddings import HuggingFaceEmbeddings
import os
from openai import OpenAI

# FIX: Swap context_precision out for context_utilization (which
requires NO reference answer!)
# Try importing capitalized versions, as metrics are often classes
from ragas.metrics import Faithfulness, AnswerRelevancy,
ContextUtilization

# Setup the native structured output judge via Groq endpoints
groq_client = OpenAI(
    api_key=os.environ.get("GROQ_API_KEY"),
    base_url="https://api.groq.com/openai/v1"
)
```

```

evaluator_llm = llm_factory(
    model="llama-3.3-70b-versatile",
    client=groq_client
)

evaluator_embeddings = HuggingFaceEmbeddings(
    model="sentence-transformers/all-MiniLM-L6-v2"
)

# Render the dataset containing user_input, response, and
# retrieved_contexts
ragas_eval_dataset = EvaluationDataset.from_hf_dataset(eval_dataset)

# Update target list to match (now using capitalized class names)
# Initialize metrics without llm/embeddings, as they will be passed to
# evaluate() function
metrics = [
    Faithfulness(llm=evaluator_llm),
    AnswerRelevancy(llm=evaluator_llm,
embeddings=evaluator_embeddings),
    ContextUtilization(llm=evaluator_llm)
]

# The previous loop to bind elements is no longer needed here if
# passed to evaluate()
# for metric in metrics:
#     metric.llm = evaluator_llm
#     if hasattr(metric, 'embeddings'):
#         metric.embeddings = evaluator_embeddings

print("\nRunning RAGAS evaluation on Groq (Llama 3.3-70B is
grading)....")

# Run default execution cleanly without reference columns, passing llm
and embeddings globally
eval_result = evaluate(
    dataset=ragas_eval_dataset,
    metrics=metrics,
)

# Convert results matrix safely
results_df = eval_result.to_pandas()
print("\n=== OVERALL RAGAS SCORES ===")
print(eval_result)

# View metric alignment output data
results_df[["user_input", "faithfulness", "answer_relevancy",
"context_utilization"]]

```

```

/tmp/ipykernel_10813/2406484305.py:24: DeprecationWarning: Importing
Faithfulness from 'ragas.metrics' is deprecated and will be removed in
v1.0. Please use 'ragas.metrics.collections' instead. Example: from
ragas.metrics.collections import Faithfulness
    from ragas.metrics import Faithfulness, AnswerRelevancy,
ContextUtilization
/tmp/ipykernel_10813/2406484305.py:24: DeprecationWarning: Importing
AnswerRelevancy from 'ragas.metrics' is deprecated and will be removed
in v1.0. Please use 'ragas.metrics.collections' instead. Example: from
ragas.metrics.collections import AnswerRelevancy
    from ragas.metrics import Faithfulness, AnswerRelevancy,
ContextUtilization
/tmp/ipykernel_10813/2406484305.py:24: DeprecationWarning: Importing
ContextUtilization from 'ragas.metrics' is deprecated and will be
removed in v1.0. Please use 'ragas.metrics.collections' instead.
Example: from ragas.metrics.collections import ContextUtilization
    from ragas.metrics import Faithfulness, AnswerRelevancy,
ContextUtilization

```

```

{"model_id": "70b90fb8ac6f499ca3dea0321a872fb5", "version_major": 2, "vers
ion_minor": 0}

```

Running RAGAS evaluation on Groq (Llama 3.3-70B is grading)....

```

{"model_id": "0ae39c537324473d8310e51c2051fe6c", "version_major": 2, "vers
ion_minor": 0}

```

WARNING:ragas.prompt.pydantic\_prompt:LLM returned 1 generations instead of requested 3. Proceeding with 1 generations.

ERROR:ragas.executor:Exception raised in Job[1]:  
AttributeError('HuggingFaceEmbeddings' object has no attribute 'embed\_query')

WARNING:ragas.prompt.pydantic\_prompt:LLM returned 1 generations instead of requested 3. Proceeding with 1 generations.

ERROR:ragas.executor:Exception raised in Job[4]:  
AttributeError('HuggingFaceEmbeddings' object has no attribute 'embed\_query')

WARNING:ragas.prompt.pydantic\_prompt:LLM returned 1 generations instead of requested 3. Proceeding with 1 generations.

ERROR:ragas.executor:Exception raised in Job[7]:  
AttributeError('HuggingFaceEmbeddings' object has no attribute 'embed\_query')

=== OVERALL RAGAS SCORES ===

```

{'faithfulness': 0.8889, 'answer_relevancy': nan,
'context_utilization': 0.4722}

```

```

{"summary": "{\n  \"name\": \"results_df[[\\\\"user_input\\\"",
\\\\"faithfulness\\\"", \\\\"answer_relevancy\\\"", \\\\"context_utilizatio

```

```

n\\\\"\\",\\n  \\\"rows\\\": 3,\\n  \\\"fields\\\": [\\n      {\\n          \\\"column\\\":
\\\"user_input\\\",\\n          \\\"properties\\\": {\\n              \\\"dtype\\\":
\\\"string\\\",\\n              \\\"num_unique_values\\\": 3,\\n              \\\"samples\\\":
[\\n                  \\\"What are the core components of an Agentic AI
architecture?\\\",\\n                  \\\"What specific challenges or bottlenecks
regarding Agentic AI are discussed?\\\",\\n                  \\\"How does memory
play a role in agent frameworks according to the papers?\\\"\\n
\\n              ],\\n              \\\"semantic_type\\\": \\\"\\\",\\n
\\\"description\\\": \\\"\\\"\\n          }\\n      },\\n      {\\n          \\\"column\\\":
\\\"faithfulness\\\",\\n          \\\"properties\\\": {\\n              \\\"dtype\\\":
\\\"number\\\",\\n              \\\"std\\\": 0.1924500897298753,\\n              \\\"min\\\":
0.6666666666666666,\\n              \\\"max\\\": 1.0,\\n
\\\"num_unique_values\\\": 2,\\n              \\\"samples\\\": [\\n
0.6666666666666666,\\n              1.0\\n          ],\\n
\\\"semantic_type\\\": \\\"\\\",\\n          \\\"description\\\": \\\"\\\"\\n      }\\n
\\n  },\\n  {\\n      \\\"column\\\": \\\"answer_relevancy\\\",\\n
\\\"properties\\\": {\\n          \\\"dtype\\\": \\\"number\\\",\\n          \\\"std\\\":
null,\\n          \\\"min\\\": null,\\n          \\\"max\\\": null,\\n
\\\"num_unique_values\\\": 0,\\n          \\\"samples\\\": [],\\n
\\\"semantic_type\\\": \\\"\\\",\\n          \\\"description\\\": \\\"\\\"\\n      }\\n
\\n  },\\n  {\\n      \\\"column\\\": \\\"context_utilization\\\",\\n
\\\"properties\\\": {\\n          \\\"dtype\\\": \\\"number\\\",\\n          \\\"std\\\":
0.42763345326585867,\\n          \\\"min\\\": 0.0,\\n          \\\"max\\\":
0.8333333332916666,\\n          \\\"num_unique_values\\\": 3,\\n
\\\"samples\\\": [],\\n          \\\"semantic_type\\\": \\\"\\\",\\n
\\\"description\\\": \\\"\\\"\\n      }\\n  }\\n  ]\\n}\\",\"type\":\"dataframe\"}

```

## Results from chunking strategy 2

```

import sys
from types import ModuleType

# 1. Keep the VertexAI dummy module mock at the very top to bypass
that legacy bug
mock_vertex = ModuleType("langchain_community.chat_models.vertexai")
mock_vertex.ChatVertexAI = None
sys.modules["langchain_community.chat_models.vertexai"] = mock_vertex

mock_llm_vertex = ModuleType("langchain_community.llms.vertexai")
mock_llm_vertex.VertexAI = None
sys.modules["langchain_community.llms.vertexai"] = mock_llm_vertex

# =====
# RAGAS 0.4+ Modern Native Open-Source Infrastructure
# =====
from ragas import evaluate, EvaluationDataset
from ragas.llms import llm_factory
from ragas.embeddings import HuggingFaceEmbeddings

```

```

import os
from openai import OpenAI

# FIX: Swap context_precision out for context_utilization (which
# requires NO reference answer!)
# Try importing capitalized versions, as metrics are often classes
from ragas.metrics import Faithfulness, AnswerRelevancy,
ContextUtilization

# Setup the native structured output judge via Groq endpoints
groq_client = OpenAI(
    api_key=os.environ.get("GROQ_API_KEY"),
    base_url="https://api.groq.com/openai/v1"
)

evaluator_llm = llm_factory(
    model="llama-3.3-70b-versatile",
    client=groq_client
)

evaluator_embeddings = HuggingFaceEmbeddings(
    model="sentence-transformers/all-MiniLM-L6-v2"
)

# Render the dataset containing user_input, response, and
# retrieved_contexts
ragas_eval_dataset = EvaluationDataset.from_hf_dataset(eval_dataset)

# Update target list to match (now using capitalized class names)
# Initialize metrics without llm/embeddings, as they will be passed to
# evaluate() function
metrics = [
    Faithfulness(llm=evaluator_llm),
    AnswerRelevancy(llm=evaluator_llm,
embeddings=evaluator_embeddings),
    ContextUtilization(llm=evaluator_llm)
]

# The previous loop to bind elements is no longer needed here if
# passed to evaluate()
# for metric in metrics:
#     metric.llm = evaluator_llm
#     if hasattr(metric, 'embeddings'):
#         metric.embeddings = evaluator_embeddings

print("\nRunning RAGAS evaluation on Groq (Llama 3.3-70B is
grading)....")

# Run default execution cleanly without reference columns, passing llm
and embeddings globally

```

```

eval_result = evaluate(
    dataset=ragas_eval_dataset,
    metrics=metrics,
)

# Convert results matrix safely
results_df = eval_result.to_pandas()
print("\n=== OVERALL RAGAS SCORES ===")
print(eval_result)

# View metric alignment output data
results_df[["user_input", "faithfulness", "answer_relevancy",
"context_utilization"]]

/tmp/ipykernel_4158/2406484305.py:24: DeprecationWarning: Importing
Faithfulness from 'ragas.metrics' is deprecated and will be removed in
v1.0. Please use 'ragas.metrics.collections' instead. Example: from
ragas.metrics.collections import Faithfulness
    from ragas.metrics import Faithfulness, AnswerRelevancy,
ContextUtilization
/tmp/ipykernel_4158/2406484305.py:24: DeprecationWarning: Importing
AnswerRelevancy from 'ragas.metrics' is deprecated and will be removed
in v1.0. Please use 'ragas.metrics.collections' instead. Example: from
ragas.metrics.collections import AnswerRelevancy
    from ragas.metrics import Faithfulness, AnswerRelevancy,
ContextUtilization
/tmp/ipykernel_4158/2406484305.py:24: DeprecationWarning: Importing
ContextUtilization from 'ragas.metrics' is deprecated and will be
removed in v1.0. Please use 'ragas.metrics.collections' instead.
Example: from ragas.metrics.collections import ContextUtilization
    from ragas.metrics import Faithfulness, AnswerRelevancy,
ContextUtilization

{"model_id": "6ab2314321ab4e9f8638ed63fddf7157", "version_major": 2, "vers
ion_minor": 0}

```

Running RAGAS evaluation on Groq (Llama 3.3-70B is grading)....

```

{"model_id": "95965d949dc54b9cb4c77e9eeceab0ff", "version_major": 2, "vers
ion_minor": 0}

```

WARNING:ragas.prompt.pydantic\_prompt:LLM returned 1 generations instead of requested 3. Proceeding with 1 generations.

ERROR:ragas.executor:Exception raised in Job[1]:  
AttributeError('HuggingFaceEmbeddings' object has no attribute 'embed\_query')

WARNING:ragas.prompt.pydantic\_prompt:LLM returned 1 generations instead of requested 3. Proceeding with 1 generations.

ERROR:ragas.executor:Exception raised in Job[4]:  
AttributeError('HuggingFaceEmbeddings' object has no attribute

```
'embed_query')
WARNING:ragas.prompt.pydantic_prompt:LLM returned 1 generations
instead of requested 3. Proceeding with 1 generations.
ERROR:ragas.executor:Exception raised in Job[7]:
AttributeError('HuggingFaceEmbeddings' object has no attribute
'embed_query')
```

=== OVERALL RAGAS SCORES ===

```
{'faithfulness': 0.9167, 'answer_relevancy': nan,
'context_utilization': 0.0000}
```

```
{"summary":{"\n  \"name\": \"results_df[[\\\"user_input\\\",
\\\"faithfulness\\\", \\\"answer_relevancy\\\", \\\"context_utilizatio
n\\\"]]\",\n  \"rows\": 3,\n  \"fields\": [\n    {\n      \"column\":
\"user_input\",\n      \"properties\": {\n        \"dtype\":
\"string\",\n        \"num_unique_values\": 3,\n        \"samples\":
[\n          \"What are the core components of an Agentic AI
architecture?\",\n          \"What specific challenges or bottlenecks
regarding Agentic AI are discussed?\",\n          \"How does memory
play a role in agent frameworks according to the papers?\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\",\n        \"column\":
\"faithfulness\",\n        \"properties\": {\n          \"dtype\":
\"number\",\n          \"std\": 0.14433756729740646,\n          \"min\":
0.75,\n          \"max\": 1.0,\n          \"num_unique_values\": 2,\n          \"samples\": [\n            1.0,\n            0.75\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        },\n        \"column\": \"answer_relevancy\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\":
null,\n          \"min\": null,\n          \"max\": null,\n          \"num_unique_values\": 0,\n          \"samples\": [],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        },\n        \"column\": \"context_utilization\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\":
0.0,\n          \"min\": 0.0,\n          \"max\": 0.0,\n          \"num_unique_values\": 1,\n          \"samples\": [],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        }\n      }\n    ]\n  },\n  \"type\": \"dataframe\"}
```